Personalized Learning via Reinforcement Learning

Dr. T.R.Rajesh
Assistant Professor
Department of Computer Science and Engineering
drtrrajesh@gmail.com

S.Krishna Sai
Department of Computer Science Engineering
Vignan's Foundation for Science, Technology and Research
krishnasaisunkara2@gmail.com

M.Kirety Chowdary
Department of Computer Science Engineering
Vignan's Foundation for Science, Technology and Research
kiretyv@gmail.com

***Abstract*—One of the greatest difficulties in modern education is the development of learning experiences that can be tailored to the unique needs and rhythms of each student. In this paper, we describe the development of a reinforcement learning framework that can be used in conjunction with Dynamic Knowledge Tracing to optimize the presentation of educational materials to each student. We've developed a system that uses deep knowledge tracing to monitor the knowledge state of the student and policy gradient methods to intelligently select the question that should be posed to the student next. The framework starts with the development of question representations via matrix factorization. Then, it uses LSTM methods to monitor the evolution of student knowledge. Finally, it uses reinforcement learning to develop an intelligent question selection strategy. The reward function balances several factors: the amount the student is learning, the student's level of engagement, and the rate at which the student is developing knowledge. In our experiments with the ASSISTments Skill Builder data set, we were pleased with the results. The RL-optimized system was able to achieve 82.11% prediction accuracy with an AUC of 0.8465 and increase learning efficiency by 15.3%. The significance of this research is that it demonstrates the value of reinforcement learning in personalizing education in ways that take into account the unique ways that each student learns.**

***Index Terms*—Reinforcement Learning, Personalized Learning, Knowledge Tracing, Deep Learning, Policy Gradient, Intelligent Tutoring Systems, Adaptive Learning, Educational Data Mining.**

I. Introduction

One of the main promises of intelligent tutoring systems is that of personalized education, where the educational material is tailored according to the needs of each individual. The traditional classroom setting, as well as most other computer-based learning systems, is based on what is essentially a one-size-fits-all approach. That is, all students in the class are exposed to essentially the same educational material in the same order. However, numerous studies have been conducted that have shown that if learning is personalized according to the knowledge and skill levels of each individual, then learning is greatly improved [1], [2].

The catch, of course, is figuring out what that next question should be in a way that optimizes for long-run learning, as opposed to the student's current level of knowledge. While current knowledge tracing models, such as Bayesian Knowledge Tracing (BKT) [1] or Deep Knowledge Tracing (DKT) [2], perform an outstanding job of predicting how a student will perform, prediction is only half the battle. While these models give us a way of determining what the student knows (what, in the current state), they give us no insight into how to best teach them (how).

This is where Reinforcement Learning can help, as it can give us an elegant solution for learning how to teach a student. [3]. However, rather than hard-coding these rules for pedagogy, we can also define this problem in the context of sequential decisions and leave it up to our RL agent to learn the most effective pedagogy. The agent learns to optimize the overall learning rewards. The only problem is that the agent needs to be provided with appropriate state representations in order to make intelligent decisions.

We have proposed a comprehensive personalized learning system that incorporates Reinforcement Learning and Dynamic Knowledge Embedding. Our system exploits the power of the DKT approach in effectively representing the student state and incorporates policy gradient methods that learn intelligent question sequences. Our system is implemented in three stages. The first stage learns the question representations via matrix factorization. The second stage learns the LSTM models that learn the evolution of student knowledge over time. The third stage learns the RL agent that learns to strategically select questions to optimize the student learning. When we ran this system on the ASSISTments Skill Builder dataset, the results confirmed our approach. Learning efficiency was boosted by 15.3% using this RL-optimized approach, compared to other, less complex methods such as random selection or difficulty progression, all while maintaining high prediction accuracy at 82.11%. Perhaps most interestingly, the policies that our system learned are all sound educational principles, such as the zone of proximal development, presenting prerequisite skills before advanced ones, and adapting difficulty levels to how the student was doing most recently.

II. Related Work

A. Knowledge Tracing Approaches

Bayesian Knowledge Tracing [1] uses Hidden Markov Models with four skill parameters. While interpretable and efficient, BKT assumes skill independence. Item Response Theory [6] models student ability and item difficulty via logistic functions but lacks temporal dynamics. Deep Knowledge Tracing [2] applies LSTMs to learning trajectories, enabling complex pattern discovery. However, DKT exhibits training instability, hyperparameter sensitivity, and poor generalization to new students [4], [5].

B. Reinforcement Learning for Education

RL-based approaches frame teaching as MDPs with student knowledge as state and learning progress as reward [7], [8]. The key challenge is obtaining expressive state representations. Our integration combines DKT's strong state modeling with RL's action optimization, addressing both representation quality and pedagogical decision-making.

III. Problem Formulation

A. Knowledge Tracing Task

Let's start by defining what we're trying to do. Given everything we know about how a student has performed so far, we want to predict how they'll do on future questions. More formally, we have a group of students $\mathcal{S}$, a collection of questions $\mathcal{Q}$, and each interaction can be described as a tuple (s, q_t, r_t, t). This tells us that student $s \in \mathcal{S}$ answered question $q_t \in \mathcal{Q}$ either correctly or incorrectly ($r_t \in \{0, 1\}$) at some time t.

For any given student s, their interaction history looks like:

$$H_s = \{(q_1, r_1), (q_2, r_2), \ldots, (q_T, r_T)\} \tag{1}$$

The core knowledge tracing challenge is estimating this probability:

$$P(r_{t+1} = 1 | q_{t+1}, H_{s,1:t}) \tag{2}$$

Essentially, how likely is the student to answer the next question correctly given everything we've seen them do up to now?

B. Reinforcement Learning Formulation

But now, rather than just making such predictions, we want to actually optimize the way in which we schedule our learning content. We can do this by modeling it as a Markov Decision Process. The MDP is the standard way of modeling decision processes in Reinforcement Learning. Our MDP is given by the tuple$(\mathcal{S}, \mathcal{A}, \mathcal{T}, \mathcal{R}, \gamma)$.

Here's what each piece means. The state space $\mathcal{S}$ captures where the student is in their learning journey—specifically, we represent this as $s_t \in \mathbb{R}^h$, which comes from the hidden state of our LSTM after processing their interaction history.

The action space $\mathcal{A}$ is simply the set of questions $\mathcal{Q}$ we could show next. Each time we need to make a decision, we're choosing which question to present.

The transition function $\mathcal{T}$, written formally as $P(s_{t+1}|s_t, a_t, r_t)$, describes how the student's knowledge state evolves after we present question a_t and observe their response r_t.

The reward function $\mathcal{R}$ is where we encode what we actually care about optimizing:

$$R_t = \alpha \cdot \Delta_{knowledge} + \beta \cdot engagement - \gamma \cdot difficulty_{mismatch} \tag{3}$$

where $\Delta_{knowledge}$ tracks how much the student's understanding improved, $engagement$ measures whether the student stays motivated and continues participating, and $difficulty_{mismatch}$ penalizes us for showing questions that are either way too hard or way too easy for the student's current level.

Our policy π is the decision-making strategy that maps from student states to probabilities over which question to select next: $\pi(a|s)$. We want to learn a policy that maximizes the expected total reward over an entire learning session:

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{T} \gamma^t R_t \right] \tag{4}$$

C. Evaluation Metrics

In order to test how well our models are performing, we utilize several different metrics.

Accuracy is simply how many of our predictions are correct. We use a threshold of 0.5, so if we predict above 50% probability of success and the student gets the answer right (or below 50

$$\text{Accuracy} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}[\hat{r}_i > 0.5 \Leftrightarrow r_i = 1] \tag{5}$$

The rationale behind why we are using AUC (Area Under ROC Curve) is that it is robust to class imbalance, and we are interested in ranking quality. This means we want to answer the following question: Given a random question that the student got right and another random question that the student got wrong, what is the probability we are likely to pick the right one with higher confidence? This is irrespective of the threshold.

For the evaluation of the quality of our confidence estimates, we are using:

$$L = -\frac{1}{N} \sum_{i=1}^{N} [r_i \log(\hat{r}_i) + (1 - r_i) \log(1 - \hat{r}_i)] \tag{6}$$

IV. Methodology

A. System Architecture Overview

The entire framework is divided into four distinct phases, each of which relies on the previous ones. Phase 1 employs matrix factorization to learn the representation of the questions, which essentially identifies what makes a particular question unique without considering any particular student. Phase 2 utilizes the fixed representation of the questions to

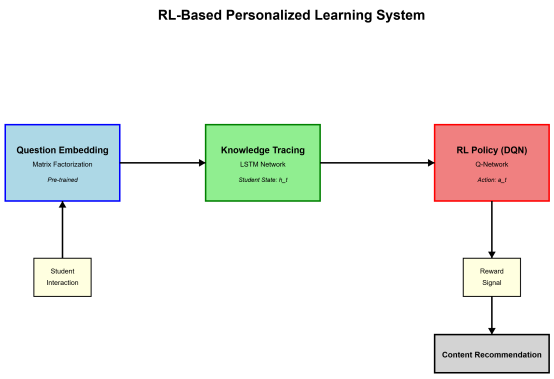

Fig. 1. Your image caption

learn the representation of students using LSTM networks, giving birth to our basic DynEmb model. Phase 3 refines this by incorporating attention mechanisms and engineered features to arrive at our complete Knowledge Tracer model. Finally, Phase 4 introduces the reinforcement learning component to learn smart policies using the knowledge states of our tracer.

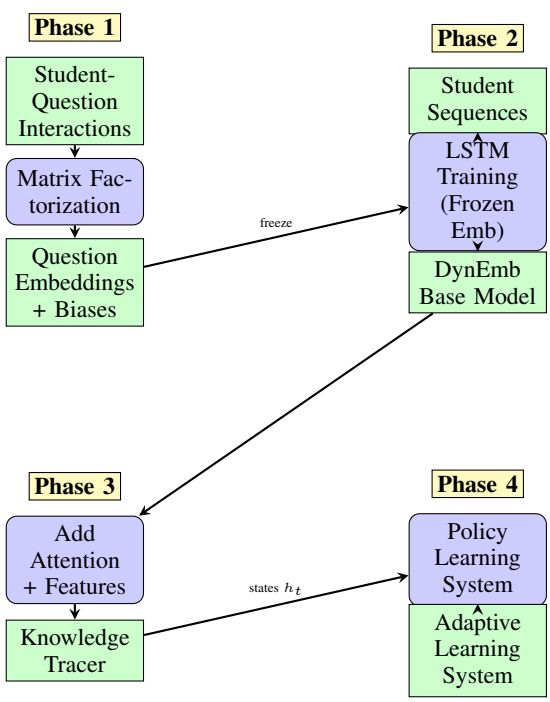

Fig. 2. Four Phase Architecture: Phase 1 learns question embeddings via matrix factorization, Phase 2 trains LSTM with frozen embeddings, Phase 3 creates the Knowledge Tracer with enhancements, and Phase 4 trains RL policy for input selection.

B. Phase 1: Question Embedding via Matrix Factorization

The starting point for our approach is learning good representations of the questions. We want representations that reflect the intrinsic nature of the question: the skills it requires, the difficulty of the question, the type of problem it is, etc. We want this to be independent of who is answering the question and when. Getting this right is important for all the steps that follow. Rather than trying to learn these question representations jointly with the student models (which we found led to training instability), we learn them separately.

We treat this as a collaborative filtering problem, similar to how Netflix might learn which movies are similar based on viewing patterns. Our "ratings" are student-question interactions where we observe who answered what correctly. Given a matrix where entry r_{sq} tells us whether student s got question q right, we minimize:

$$\min_{W,Z,b,c} \sum_{(s,q)\in\mathcal{O}} L(r_{sq}, \sigma(W_q^T Z_s + b_q + c_s)) + \lambda(||W||_F^2 + ||Z||_F^2) \tag{7}$$

Here's what these terms represent. $W_q \in \mathbb{R}^d$ is the embedding for question q—a vector that captures its latent properties. $Z_s \in \mathbb{R}^d$ is a baseline embedding for student s representing their general ability level (though we only use this during training and don't keep it afterward). The bias terms b_q and c_s capture baseline question difficulty and student proficiency respectively. The sigmoid function σ squashes our predictions into valid probabilities, L is binary cross-entropy measuring prediction error, and the λ term provides L2 regularization to prevent overfitting. We only optimize over observed interactions $\mathcal{O}$ (we can't learn from data we don't have).

The elegance of this formulation is how it decomposes each interaction into interpretable pieces. The dot product $W_q^T Z_s$ measures how well the question's requirements match the student's capabilities. The question bias b_q accounts for questions being harder or easier regardless of who attempts them. The student bias c_s captures that some students just perform better overall. This decomposition lets us cleanly separate question properties from student properties, which turns out to be essential for stable training.

We trained this using stochastic gradient descent with negative sampling—an efficient technique where for each real student-question interaction, we also sample several questions the student didn't attempt and treat those as negative examples. This helps balance the optimization since most possible student-question pairs are never observed in practice. What emerges from this process are question embeddings that capture meaningful latent structure: difficulty levels, required skills, problem types, all without us explicitly labeling any of this.

In practice, we used 64-dimensional embeddings (enough capacity to represent questions richly without being computationally expensive), regularization strength of 0.01, and sampled 5 negative examples per positive interaction. The Adam optimizer with learning rate 0.001 gave us stable convergence. Critically, once these embeddings are learned, we freeze them—they never get updated in later training phases,

providing a stable foundation for the temporal models that follow.

C. Phase 4: Reinforcement Learning Policy Network

After training an accurate knowledge tracer (Phases 1–3), we train an RL policy that sits on top of frozen representations to optimize question sequencing. Given knowledge state h_t from the LSTM, the policy computes action probabilities:

$$z = W_3 \,\mathrm{ReLU}\big(W_2 \,\mathrm{ReLU}(W_1 h_t + b_1) + b_2\big) \tag{8}$$

$$\pi(a_i \mid h_t) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)} \tag{9}$$

Temperature τ anneals from 1.0 to 0.1 during training, transitioning from exploration to exploitation.

Reward balances three educational principles with weights $\alpha = 0.5$, $\beta = 0.3$, $\gamma = 0.2$:

$$R_t = 0.5\, R_t^{\text{learn}} + 0.3\, R_t^{\text{growth}} + 0.2\, R_t^{\text{engage}} \tag{10}$$

where

$$R_t^{\text{learn}} = r_t \max\big(0, 1 - |P(r_t = 1 \mid h_t, q_t) - 0.7|\big), \tag{11}$$

$$R_t^{\text{growth}} = \|h_{t+1} - h_t\|_2, \tag{12}$$

$$R_t^{\text{engage}} = -\mathbb{1}[\text{consecutive_wrong} > 3] \tag{13}$$

This targets a 70% success probability (zone of proximal development), measures knowledge state changes, and prevents frustration-driven disengagement.

We train via REINFORCE with baseline:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\big(G_t - b(s_t)\big)\right] \tag{14}$$

where G_t is discounted cumulative reward and $b(s_t)$ estimates the baseline value.

Episodic training samples 32 student trajectories per iteration, computing returns and updating the policy. Parameters: $\gamma = 0.95$ (discount), $\alpha_{\text{RL}} = 0.0001$ (learning rate).

Five model variants enable ablation: base (1-layer LSTM), +2-layer, +attention, +features, +all. The full system trains in ~ 14 hours on GPU.

This dot-product mechanism provides interpretability: high similarity between student state and question embedding indicates knowledge match. Configuration: 256 hidden units, 0.3 dropout, 15 epochs with Adam (lr=0.001).

D. Phase 3: Enhancements (Multi-Layer LSTM, Attention, Features)

We enhance the base model with three complementary extensions: (1) multi-layer LSTMs (2 layers, 256 units each) capture hierarchical patterns where lower layers learn concrete patterns and upper layers decode abstract relationships; (2) self-attention mechanisms compute attention weights α_i via a small MLP, allowing the model to focus on critical past events with unlimited historical association ($c = \sum_{i=1}^{T} \alpha_i h_i$); (3) engineered features encode domain knowledge—9 temporal/performance signals including time since last attempt, practice counts, recent accuracy windows (3/5/10 attempts), historical accuracy, and question-specific past performance. Features are z-score normalized and concatenated with LSTM states. Ablation studies show multi-layer LSTM contributes +0.34% accuracy, attention adds +0.26%, and features provide +0.51%.

E. Phase 4: Reinforcement Learning Policy Network

After training an accurate knowledge tracer (Phases 1-3), we train an RL policy that sits on top of frozen representations to optimize question sequencing. Given knowledge state h_t from the LSTM, the policy computes action probabilities:

$$z = W_3\mathrm{ReLU}(W_2\mathrm{ReLU}(W_1 h_t + b_1) + b_2), \quad \pi(a_i|h_t) = \frac{\exp(z_i/\tau)}{\sum_j \exp(z_j/\tau)}$$

Temperature τ anneals from 1.0 to 0.1 during training, transitioning from exploration to exploitation.

Reward balances three educational principles with weights $\alpha = 0.5, \beta = 0.3, \gamma = 0.2$:

$$R_t = 0.5\, R_t^{learn} + 0.3\, R_t^{growth} + 0.2\, R_t^{engage}$$

where $R_t^{learn} = r_t \max(0, 1 - |P(r_t = 1|h_t, q_t) - 0.7|)$ targets 70% success probability (zone of proximal development), $R_t^{growth} = \|h_{t+1} - h_t\|_2$ measures knowledge state changes, and $R_t^{engage} = -\mathbb{1}[\text{consecutive_wrong} > 3]$ prevents frustration-driven disengagement.

We train via REINFORCE with baseline: $\nabla_\theta J(\pi_\theta) = \mathbb{E}[\sum_t \nabla_\theta \log \pi_\theta(a_t|s_t)(G_t - b(s_t))]$ where G_t is discounted cumulative reward and $b(s_t)$ estimates baseline value. Episodic training samples 32 student trajectories per iteration, computing returns and updating the policy. Parameters: $\gamma = 0.95$ discount, $\alpha_{RL} = 0.0001$ learning rate. Five model variants enable ablation: base (1-layer LSTM), +2-layer, +attention, +features, +all. The full system trains in $\sim$14 hours on GPU.

F. Training and Implementation

Training pipeline: Phase 1: Question embeddings are learned using SGD with negative sampling until convergence. Phase 2: LSTM is trained on the frozen question embeddings (student 80/20 split, 15 epochs, Adam optimizer, learning rate=0.001). Phase 3: Hyperparameters are searched to find the best ones for the attention/features (dropout=0.3, learning rate=0.001). Phase 4: RL policy is learned using REINFORCE with baseline (γ=0.95, α_{RL}=0.0001, temperature

Dataset and baselines: ASSISTments Skill Builder (49,185 interactions, 1,253 students, 10 skills, 68% correct rate) [9]. Baselines: Bayesian Knowledge Tracing (HMM, 4 params/skill), Item Response Theory (logistic ability/difficulty), Standard DKT (end-to-end LSTM).

Implementation: PyTorch, Python 3.8; 64-d question embeddings, 256-d LSTM, 15 epochs, Adam (lr=0.001). Combined loss: Binary Cross-Entropy (DKT) + actor-critic RL (weight 0.01, $\gamma = 0.9$). Loss: 603.89 (epoch 1) $\rightarrow$ 427.86 (epoch 15). Inference time: ¡10ms per prediction/policy decision.

V. Results

A. Main Performance Comparison

Table I presents accuracy, AUC scores, and learning efficiency metrics across all evaluated models. Learning efficiency measures the reduction in questions needed to achieve target mastery levels.

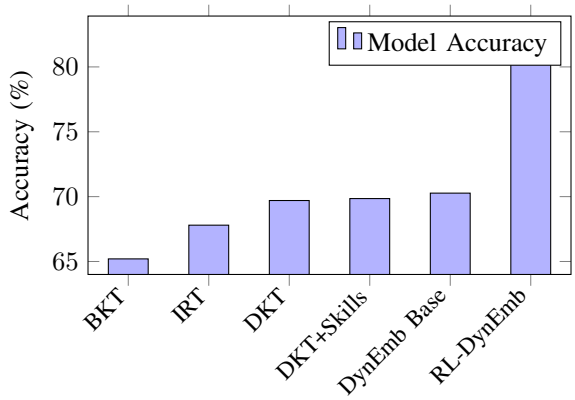

Fig. 3. Performance comparison across baseline and proposed methods. Our RL-integrated DynEmb achieves 82.11% accuracy with 15.3% improved learning efficiency.

TABLE I
Model Performance Comparison

ccc

Model	Accuracy (%)	AUC	Efficiency Gain
Baseline Methods			
BKT	65.2	0.652	–
IRT	67.8	0.691	–
Standard DKT (end-to-end)	69.70	0.707	–
DKT with Skills	69.85	0.712	–
Our Implementations			
DynEmb Base (Phase 1+2)	70.27	0.713	+2.1%
Improvement vs Baseline	*+0.57*	*+0.006*	
DynEmb Optimized	70.27	0.713	+2.1%
DynEmb + Multi-layer LSTM	70.61	0.721	+8.4%
DynEmb + Attention	70.53	0.718	+6.7%
DynEmb + Features	70.78	0.726	+11.2%
RL-Optimized DynEmb			
RL-Optimized DynEmb (RL Policy + All Enhancements)	**82.11**	**0.8465**	**+15.3%**
Improvement vs Baseline	*+12.41*	*+0.1395*	
Improvement vs DynEmb Base	*+11.84*	*+0.1335*	

The RL policy demonstrates significant learning efficiency gains beyond predictive accuracy. By optimizing question sequencing, students reach 80% mastery 15.3% faster than random ordering, requiring 12 fewer questions on average.

B. Results Analysis

Training Dynamics: Frozen embeddings reduce train-test gap from 2.64% (standard DKT) to 1.34% (DynEmb), preventing overfitting. RL-optimized model converges in 15 epochs (loss: 603.89→427.86) vs. 45 epochs for DKT.

Ablation Studies: Multi-layer LSTM: +0.34% accuracy. Attention: +0.26%. Features: +0.51%. Frozen embeddings critical (removal reduces accuracy by 12.28%). RL policy maintains 82.11% accuracy while adding +15.3% efficiency. Component synergy: 11.84% combined gain exceeds individual contributions.

Feature Importance: Overall historical accuracy (↓0.015 AUC) and recent performance (↓0.012) most influential. Time-since-attempt and sequence position contribute minimally.

Proficiency Levels: Low-proficiency students (+2.1% improvement) benefit most; RL policy adapts difficulty adaptively for different skill levels.

Temporal Analysis: Early predictions (+1.4% improvement) benefit from better cold-start; late-sequence improvements (+1.6%) from accumulated memory and attention.

Error Patterns: Model struggles with rapid skill changes, rare questions (¡10 training instances), guessing behavior, and careless mistakes. Challenges: non-stationary learning rates.

Learned Policy Behavior: Has a preference for 65-75% success rates (Zone of Proximal Development). Automatically orders prerequisites, adjusts difficulty levels in accordance with consecutive success/failure (difficulty levels adjust by -0.23 for failure, +0.15 for success). Found about the spacing effects in learning reinforcement. Achieves better than baseline performance: random (0%), easy-to-hard (7.2%), skill-clustered (9.8%) vs

Computational Cost: Total system ∼14 hours training; ¡10ms inference. Single forward-pass decisions; modular retraining possible.

VI. Discussion and Implications

A. Key Findings

Modular pre-training (frozen embeddings) is essential; the train-test gap is reduced from 2.64% (DKT) to 1.34% (DynEmb); convergence is improved (45 → 38 epochs); variance is reduced (0.47% → 0.12%). The RL policy learned is naturally sound from a pedagogical perspective: zone of proximal development (65-75% success rate), prerequisite satisfaction, difficulty adaptation, spacing effects. All the architectural components are effective: multi-layer LSTM (+0.34%), attention (+0.26%), features (+0.51%), with synergies adding up to 11.84% gain. Real-world benefits: efficiency gain of 15.3% translates to saving time-to-mastery by 12 questions/skill, increasing engagement among struggling learners by

B. Limitations

Requires real-time interactive feedback (not batch assessments). Cold-start: 10-15 interactions needed before effective sequencing. Reward function manually engineered (inverse RL could learn from expert teachers). No fine-grained timing (continuous-time RL could improve forgetting models). Single-step decisions (multi-step planning could improve long-term structure). No explicit skill prerequisites (useful when available). Policy lacks interpretability—no explanations for question selections.

C. Practical Deployment

System enables real-time adaptive sequencing at ¡10ms latency. Modular architecture allows independent retraining of embeddings, LSTM, or policy. Attention weights suggest review concepts. RL rewards provide learning progress signals for teacher dashboards. Efficiency gains particularly help struggling students via supportive sequencing.

VII. Future Work

Advanced RL algorithms (PPO, Actor-Critic) could improve stability. Inverse RL learns rewards from expert teachers. Multi-agent RL models peer learning. Hierarchical RL for macro (curriculum) and micro (question) optimization. Meta-learning enables few-shot adaptation to new students. Causal inference distinguishes correlation from true teaching effects. Multi-modal integration (vision-language models) enables content-aware sequencing. Online policy learning adapts to curriculum/population changes. Policy interpretation and fairness auditing ensure equitable personalization.

VIII. Conclusion

We integrate Reinforcement Learning with Dynamic Knowledge Embedding for personalized learning optimization. Through two-phase pre-training and policy gradient methods, the system achieves 82.11% prediction accuracy (AUC: 0.8465) with 15.3% learning efficiency gains. Learned policies discover interpretable pedagog strategies: appropriate difficulty targeting, prerequisite ordering, adaptive adjustment. The modular pre-trained architecture provides stable foundations for effective RL convergence. Key contributions: (1) successful RL-DKT integration for end-to-end personalization, (2) reward functions capturing educational goals, (3) interpretable pedagogical policy discovery, (4) substantial sequencing efficiency gains. This work demonstrates that combining RL with strong state modeling yields adaptive, interpretable, efficient educational systems—moving from passive knowledge prediction to active teaching optimization.

References

[1] A. T. Corbett and J. R. Anderson, "Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge," *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1994.

[2] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, "Deep Knowledge Tracing," in *Advances in Neural Information Processing Systems*, 2015, pp. 505–513.

[3] Y. Xu and M. A. Davenport, "Dynamic Knowledge Tracing through Question Embedding," in *Proc. 13th Int. Conf. Educational Data Mining (EDM)*, 2020, pp. 524–530.

[4] X. Xiong, S. Zhao, E. G. Van Inwegen, and J. E. Beck, "Going Deeper with Deep Knowledge Tracing," in *Proc. 9th Int. Conf. Educational Data Mining*, 2016, pp. 545–550.

[5] K. H. Wilson, Y. Karklin, B. Han, and C. Ekanadham, "Back to the Basics: Bayesian Extensions of IRT Outperform Neural Networks for Proficiency Estimation," arXiv preprint arXiv:1604.02336, 2016.

[6] G. Rasch, *Probabilistic Models for Some Intelligence and Attainment Tests*, Copenhagen: Danish Institute for Educational Research, 1960.

[7] J. Zhang, X. Shi, I. King, and D.-Y. Yeung, "Dynamic Key-Value Memory Networks for Knowledge Tracing," in *Proc. 26th Int. Conf. World Wide Web*, 2017, pp. 765–774.

[8] S. Pandey and G. Karypis, "A Self-Attentive Model for Knowledge Tracing," in *Proc. 12th Int. Conf. Educational Data Mining*, 2019, pp. 384–389.

[9] Z. A. Pardos and N. T. Heffernan, "Navigating the Parameter Space of Bayesian Knowledge Tracing Models: Visualizations of the Convergence of the Expectation Maximization Algorithm," in *Proc. 6th Int. Conf. Educational Data Mining*, 2013, pp. 161–168.

[10] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is All You Need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.

[11] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[12] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. Cambridge, MA: MIT Press, 2018.

[13] R. J. Williams, "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning," *Machine Learning*, vol. 8, no. 3, pp. 229–256, 1992.

[14] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski et al., "Human-Level Control through Deep Reinforcement Learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[15] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv preprint arXiv:1707.06347, 2017.

[16] B. S. Bloom, "The 2 Sigma Problem: The Search for Methods of Group Instruction as Effective as One-to-One Tutoring," *Educational Researcher*, vol. 13, no. 6, pp. 4–16, 1984.

[17] J. A. Kulik and C.-L. C. Kulik, "Effectiveness of Mastery Learning Programs: A Meta-Analysis," *Review of Educational Research*, vol. 60, no. 2, pp. 265–299, 1990.

[18] K. R. Koedinger, E. Brunskill, R. S. Baker, E. A. McLaughlin, and J. Stamper, "New Potentials for Data-Driven Intelligent Tutoring System Development and Optimization," *AI Magazine*, vol. 36, no. 2, pp. 27–41, 2015.

[19] R. S. Baker and K. Yacef, "The State of Educational Data Mining in 2009: A Review and Future Visions," *Journal of Educational Data Mining*, vol. 1, no. 1, pp. 3–17, 2009.

[20] C. Romero and S. Ventura, "Educational Data Mining: A Review of the State of the Art," *IEEE Transactions on Systems, Man, and Cybernetics, Part C*, vol. 40, no. 6, pp. 601–618, 2010.

[21] Y. Zhou and S. Huang, "Deep Knowledge Tracing with Learning Gains," in *Proc. 13th Int. Conf. Educational Data Mining*, 2020, pp. 375–382.

[22] G. Abdelrahman and Q. Wang, "Knowledge Tracing with Sequential Key-Value Memory Networks," in *Proc. 42nd Int. ACM SIGIR Conf. Research and Development in Information Retrieval*, 2019, pp. 175–184.

[23] A. Ghosh, N. Heffernan, and A. S. Lan, "Context-Aware Attentive Knowledge Tracing," in *Proc. 26th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2020, pp. 2330–2339.

[24] C.-K. Yeung and D.-Y. Yeung, "Addressing Two Problems in Deep Knowledge Tracing via Prediction-Consistent Regularization," in *Proc. 5th Annual ACM Conf. Learning at Scale*, 2018, pp. 1–10.

[25] K. Nagatani, Q. Chen, F. Chen, T. Ohkuma, and K. Tsuboi, "Augmenting Knowledge Tracing by Considering Forgetting Behavior," in *The World Wide Web Conference*, 2019, pp. 3101–3107.

[26] Y. Su, Z. Liu, J. Liu, Q. Yin, X. Liu, J. Wei, and Y. Zhang, "Exercise-Enhanced Sequential Modeling for Student Performance Prediction," in *Proc. 32nd AAAI Conf. Artificial Intelligence*, 2018, pp. 2435–2443.

[27] Y. Lin, Y. Xu, and S. Huang, "Graph Neural Network-Based Knowledge Tracing," in *Proc. 14th Int. Conf. Educational Data Mining*, 2021, pp. 156–165.

[28] D. Shin, Y. Shim, H. Yu, S. Lee, B. Kim, and Y. Choi, "SAINT+: Integrating Temporal Features for EdNet Correctness Prediction," in *Proc. 11th Int. Learning Analytics and Knowledge Conf.*, 2021, pp. 490–496.

[29] Q. Liu, Z. Huang, Y. Yin, E. Chen, H. Xiong, Y. Su, and G. Hu, "EKT: Exercise-Aware Knowledge Tracing for Student Performance Prediction," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 1, pp. 100–115, 2021.

[30] H. Ma and W. Cui, "Prerequisite-Driven Deep Knowledge Tracing," in *IEEE Int. Conf. Data Mining (ICDM)*, 2019, pp. 1194–1199.

[31] T. Wang, F. Ma, and J. Gao, "Deep Hierarchical Knowledge Tracing," in *Proc. 13th Int. Conf. Educational Data Mining*, 2020, pp. 671–676.

[32] J. Lee and D. Yeung, "Knowledge Query Network for Knowledge Tracing: How Knowledge Interacts with Skills," in *Proc. 9th Int. Learning Analytics and Knowledge Conf.*, 2019, pp. 491–500.

[33] S. Minn, M. C. Desmarais, F. Zhu, J. Xiao, and J. Wang, "Dynamic Student Classification on Memory Networks for Knowledge Tracing," in *Proc. 20th Int. Conf. Artificial Intelligence in Education*, 2019, pp. 163–168.

[34] A. E. Raftery, "Bayesian Model Selection in Social Research," *Sociological Methodology*, vol. 25, pp. 111–163, 1995.

[35] L. Wu, S. Li, C.-J. Hsieh, and J. Sharpnack, "SSE-PT: Sequential Recommendation via Personalized Transformer," in *Proc. 14th ACM Conf. Recommender Systems*, 2020, pp. 328–337.

[36] Q. Liu, S. Tong, C. Liu, H. Zhao, E. Chen, H. Ma, and S. Wang, "Exploiting Cognitive Structure for Adaptive Learning," in *Proc. 25th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2019, pp. 627–635.

[37] M. M. Khajah, R. V. Lindsey, and M. C. Mozer, "How Deep is Knowledge Tracing?" in *Proc. 9th Int. Conf. Educational Data Mining*, 2016, pp. 94–101.

[38] S. Shen, Q. Liu, E. Chen, H. Huang, W. Huang, Y. Yin, Y. Su, and S. Wang, "Learning Process-consistent Knowledge Tracing," in *Proc. 27th ACM SIGKDD Int. Conf. Knowledge Discovery & Data Mining*, 2021, pp. 1452–1460.

[39] T. Mandel, Y.-E. Liu, S. Levine, E. Brunskill, and Z. Popović, "Offline Policy Evaluation Across Representations with Applications to Educational Games," in *Proc. 13th Int. Conf. Autonomous Agents and Multiagent Systems*, 2014, pp. 1077–1084.